



S.G. Ganesh

Copy-paste Programming

When programmers are in a hurry to get work done, they sometimes practice 'copy-paste programming'. In this column, we take a closer look at the consequences of this practice.

A widely known secret among programmers, who are under pressure to be 'productive' and churn out hundreds and thousands of lines of code every day, is to make extensive use of two commands: `Ctrl+C` and `Ctrl+V`!

So just how much code is duplicated in real world software? We don't know about closed source software, but for open source software, a study showed that 8.7 per cent of GCC, 19 per cent of X Windows, 22.7 per cent of Linux and 29 per cent of JDK consisted of duplicate code! For any seasoned programmer, these numbers are not at all a surprise. So, instead of talking about why code duplication occurs, let's discuss the impact or consequences of copy-paste programming. Before that, let us briefly go over the kind of code duplicates (a.k.a. code clones) that abound.

Type 1 clones: An exact copy without modifications, except for white spaces, new lines and comments.

Type 2 clones: Syntactically identical copy, with only variable, type or function names changed.

Type 3 clones: A copy with modifications, such as statements changed, added or removed.

Apart from these three types of clones, we also have a fourth kind: code segments that semantically do the same thing, but are syntactically different. These clones cannot be detected automatically by clone analysers, but need to be found manually.

A common mistake while copying code is that programmers copy code, but forget to make the relevant changes necessary for the copied code to be used in the context into which it has been copied. For this reason, Type 1 and Type 2 clones can result in bugs.

All types of code clones are undesirable, but Type 3 clones (copy with modifications), also known as 'inconsistent clones', are especially prone to bugs. This is because in Type 3 clones, a code block is copied, and changes are made that are inconsistent to the original intent of the code segment. In this case, the code is syntactically correct, but semantically incorrect, resulting in bugs. For example, in Eclipse 3.2.2, the file `FeatureExportWizard.java` had code identical to code in `PluginExportWizard.java`, indicating copied code. Further, there was a statement `target.appendChild(export);` that was missing in `FeatureExportWizard.java`, which led to an incorrectly formed XML file; this problem was also filed as a bug (ID 150570).

There is another problem with code clones. If the original code segment has a bug, and if the code is copied, the bug propagates! For example, a defect in Mozilla (Bug ID 217604) is a code block containing a bug that was copied in 12 places! So if the same piece of code is copied in 10 different places, obviously it's difficult to make changes to all the duplicated code segments when fixing the problem. So, most programmers fix only the code clone piece on which a bug was raised—in all other unfixed clones, the bug lurks, only to be discovered much later. Hence, code clones can affect the reliability of applications as well, which most programmers don't understand or appreciate.

If the code is duplicated in many different places, it becomes more difficult to understand or comprehend the code. Why? The human mind can hold only a limited number of chunks or items in working memory (known as the 'Seven plus or minus two rule'), so the amount of information we can process at a time is severely limited. Because code duplication tends to 'bloat' code, it increases the complexity of the software code base. Hence, the main impact of code clones is on the maintainability of the application.

How do we know which code clones are serious, and which ones to address first? A practical way is to prioritise each set of code clones, based on the following formula:

Clone severity = the number of code duplicates * the number of lines of code duplication * the nature of code duplication (Types 1, 2, 3 or 4—the higher it is in this sequence, the more severe it is) * distribution of clones (intra-file, intra-module, inter-module—the higher, the more severe it is).

Now, how does one detect code clones? These days, real-world (open as well as closed source) software applications easily cross a million lines of code. Manually finding duplicate code segments is impossible in such large code bases, and the only practical option is to use automated clone detection tools. Given the importance of detecting code clones, it is not surprising to see a proliferation of automated tools—both commercial and open source—to detect duplicate code. For example, Simian is a commercial tool (see www.harukizaemon.com/simian) and PMD's CPD (Copy Paste Detector) (see pmd.sourceforge.net/cpd.html) is an open source tool. However, remember that clone